

The Kaiju Lab, compressed

Shared participant instructions - every scenario, same steps

Contents

- Overview and rules
- Before the session
- Section 1: Discovery
- Section 2: Security and Responsible AI
- Section 3: Research, Plan, Implement
- Show-and-tell and close

Derived from The Kaiju Lab v1.0 (Sumer Verma, 7 October 2026). Capgemini internal. The sample application is MIT licensed; the scenario is fiction and contains no client material.

Kaiju Lab, compressed: shared instructions

Same steps for every scenario. One repo holds all four scenarios, pre-seeded under `.copilot-tracking/dt/<slug>/`. You pick one scenario pack; it has the kaiju facts, your card, what you are graded on, and the slug. Where this document says `<slug>`, use the one from your pack.

Individual work. Three sections, about 25, 20 and 25 minutes. You touch discovery, security, Responsible AI and the research-plan-implement loop. You finish none of them. That is the point.

You play one role in every scenario: the incident commander. Everyone else has already been interviewed. The coach has read those interviews and will quiz you on them.

How it fits together

Three agents, in order. Each one reads what the last one wrote to `.copilot-tracking/`. Nothing carries over in chat; `/clear` between every section.

Section	Agent	Reads	Writes	You do
1 Discovery	dt-coach	Pre-seeded scope and nine interviews	Your interview, synthesis, PRD, diagram, Azure overlay under <code>dt/<slug>/</code>	Answer five questions, then one synthesis prompt
2a Security	security-planner	Your PRD and overlay, the scoping answers	Threat model and backlog under <code>security-plans/</code>	Predict three findings, fire it, compare
2b Responsible AI	rai-planner-agent	The app's Last Stand scene	Assessment and gates under <code>rai-plans/</code>	Fire it, read the verdict
3 Research, Plan	rpi-agent	Your PRD, one requirement	Research and plan under <code>research/</code> , <code>plans/</code>	Pick the requirement, amend the plan
Homework	rpi-implement, rpi-review	The plan	Code, changes log, review	Approve edits, read the verdict

The coach's nine methods

The Design Thinking Coach works through nine methods. Folder names under `dt/<slug>/` match them. In this lab you touch 2 and 3 yourself; the coach runs 4 to 7 in one prompt and marks 6, 8 and 9 skipped.

Method	Name	Output	Here
1	Scope conversations	Stakeholder map, scope, assumptions	Pre-seeded
2	Research	Interviews	Nine pre-seeded, one by you
3	Synthesis	Themes, problem definition	Your synthesis prompt
4, 5	Brainstorming, concepts	Ideas, three concepts, one chosen	Coach, same prompt
6	Prototyping	Paper prototype	Skipped, labelled

Method	Name	Output	Here
7	Design	Diagram, Azure overlay	Coach, same prompt
8, 9	Testing, iteration	Feedback, revision	Skipped, labelled

Rules

- Kaiju facts are fixed. Page 1 of your pack has everything you need; the coach holds the rest. Never invent them.
- Human facts are yours. Invent them in role, and label them.
- "I do not know" and "assume and label it" are good answers.
- Approve every permission prompt by hand. Never auto-approve.
- Nothing real goes in the workspace. Commit locally. Never push.
- `main` is the clean baseline. Work on your own branch.

Before the session

Do these the week before. Blocked on any of them, tell the facilitator before the day.

- 1 Claude Code installed with the hve-core plugin. `/plugin` lists it; typing `/hve-core:` shows commands.
- 2 Clone the lab repo: `git clone https://github.com/joelcaggemini/titan-watch-seatac`. Node 22. `npm install`, then `npm run dev`. Four kaiju visible on the map; the roster shows six.
- 3 Pick a scenario. Read its pack: page 1 and your card. Nothing else.
- 4 Pick a reasoning model by hand. Automatic selection gives shallow coaching.

On the day, before typing:

- Open Claude Code **in the repo folder**, not a parent. Run `pwd` and check. An agent writes wherever it is.
- `git checkout -b <your-name>` from main. Commit there. Never push.
- `/clear` at the start of every section. A prompt typed into the wrong agent produces confident nonsense.
- Always type the agent command, never the prompt command. `/hve-core:dt-coach`, `/hve-core:security-planner`, `/hve-core:rai-planner-agent`, `/hve-core:rpi-agent`. The prompt commands such as `security-plan-from-prd`, `rai-capture` and `rpi` fail with "needs the role loaded first". Known defect in the Claude Code conversion.
- Open `.copilot-tracking/` in your file explorer. Watch files appear. That folder is what gets reviewed, not your chat.

Section 1: Discovery

Goal. One interview, you as incident commander, against the fixed outcome. Then one synthesis that produces a PRD, a technology-neutral diagram and an Azure overlay.

Interview. /clear, then /hve-core:dt-coach, then paste the prompt on your card. The coach reads the shared scope and the nine specialist interviews, then asks you five questions built on what the specialists said. Answer as the commander: you see only what they report, you decide, they act. Outside your card's scope, say "I do not know" or "that is not my call, ask the specialist". For fictional detail, "assume and label it". When the coach writes your interview file, read it once. Commit.

Synthesis. Same session, paste:

```
Project <slug>. Read the shared scope under .copilot-tracking/dt/<slug>/,
every interview there, and my interview under .copilot-tracking/dt/<slug>-commander/.
Restate the outcome in light of the interviews, keeping the original beside it.
Summarise the problem as if hearing it for the first time. List what you still
need before a PRD. Then draft the PRD with non-functional requirements, close
gaps with labelled assumptions, and state what it rests on. Then a
technology-neutral diagram as Mermaid. Then an opinionated Azure overlay with
the minimum services, rejected alternatives recorded, and a latency budget.
Write everything under .copilot-tracking/dt/<slug>/. Run without stopping.
Label all interviews role-play.
```

While it runs. Write down one question the coach asked that you did not expect.

Done when. PRD, diagram and overlay exist. Commit.

If. Coach restarts scope: "scope is done, read coaching-state.md". Coach declares no software needed: "a software tool is expected, continue". Coach offers the canonical deck: "no, do not ask again".

Section 2: Security and Responsible AI

Goal. A severity-ranked threat model against your PRD, and a Responsible AI assessment of the app's Last Stand decision. Two terminals, in parallel.

First. Write three security findings you predict the planner will raise. Paper. Keep it.

Terminal 1, security. /clear, then /hve-core:security-planner, then:

Use the security planner to plan the production security for this solution. PRD and architecture are under .copilot-tracking/dt/<slug>/. Scoping answers are in security-scoping-answers.md in the same folder; treat every row as answered and labelled, do not ask them again. Threat-model the boundaries, map controls to OWASP, NIST and CIS, produce a backlog. Ask me only about severe or blocking issues. Run all phases without stopping. Prioritise by severity. Commit only, never push.

Approve prompts as they come. Allowed: it answers its own scoping. Not allowed: "assume I confirm everything". When done, ask for the plan summary by severity, counts per level, Critical items one line each, and its recommended next planner. Compare the Critical list with your paper.

Terminal 2, Responsible AI. /clear, then /hve-core:rai-planner-agent, then:

Run an RAI assessment on the single-save decision: the disaster-recovery AI in this app's Last Stand scene that recommends which one of two cities to save when only one can be saved. Today it outputs a single verdict with no confidence, no explanation and no audit trail, and a commander approves it under time pressure. Assess it against NIST AI RMF and treat the autonomous life-safety decision as the central risk. Use project name single-save-decision. Imagine the scoping answers, label them, run to completion unless blocking.

Read the two opening observations before replying. Do not argue the risk down; "it is only a game" is cheating. When done, read the go/no-go and the minimum conditions.

Done when. Both summaries exist. You can name one Critical threat you did not predict and one RAI minimum condition an engineer would have skipped. Commit.

Section 3: Research, Plan, Implement

Goal. One feature from your own PRD, researched and planned against the app. Implement is homework. The plan gate is the human decision in this loop; that is the part you do in the room.

Pick the feature. Open your PRD. Choose one functional requirement the sample app could plausibly carry. One screen, one rule, one feed. Not the whole PRD.

Ask, then give. /clear, /hve-core:rpi-agent, then "What do you need from me to create an RPI plan?" It lists six things. Answer with exactly those:

```
Goal: implement <the requirement, one sentence> from the PRD at
.copilot-tracking/dt/<slug>/<prd file>.
Scope: <the one component or module>. Nothing else.
Acceptance criteria: <two or three testable lines from the requirement>.
Constraints: no backend, no tracker, no cloud tenant; mock or defer and log
the gap. No tracking paths in code or commits.
Validation: the existing test, lint and build scripts.
Context: the PRD and the Azure overlay in the same folder.
Mode: research, stop for my review; plan, stop for my approval.
```

Research gate. Read the research file under .copilot-tracking/research/. Wrong? Edit the file, not the chat. Then /hve-core:rpi-plan.

Plan gate. Read the plan and its critique under .copilot-tracking/plans/ and reviews/plans/. Amend at least one item in the file: change a status, add a requirement, add a test case. Then tell the agent "plan amended in the file". Commit.

Homework. /hve-core:rpi-implement, then /hve-core:rpi-review. Bring the review verdict to the next session. If your PRD came out too thin to plan from, the Last Stand feature is pre-seeded under plans/2026-10-07/ as a fallback.

Done when. Research and plan exist, the plan carries your amendment, and the critique has run.

Show-and-tell and close

Three minutes per scenario, one presenter. Four items, one sentence each.

- 1 One coach question you did not expect.
- 2 One Critical threat you did not predict.
- 3 One plan amendment and why.
- 4 Where the loop stopped.

Room names gaps, not compliments.

Closing questions.

- Which finding would your team have missed without the planner?
- The review finds zero defects and still refuses acceptance. Would you have shipped?
- What broke today, and would it break on a client machine?

Afterwards. Everything is on your local branch. The agent wrote the commit messages. Never push.